

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#06)

Use of Venus Simulator for RISC-V Procedure Execution

Student Name: Muhammad Abdullah Tanveer Roll No: 24i-6508 Section: A

Date performed: 29.09.2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1 :

Description:
This task demonstrates how procedures affect register contents during function calls in RISC-V assembly. It identifies the problem caused by modifying a saved register and fixes it by using another register. It helps understand register preservation and the calling convention.
Code:
<pre>.text main: li a0, 0 # set this to zero li s0, 2500 jal ra, procedure # call procedure add t1, s0, a0 # result should be 1003+2500 li a0, 1 # setup system call for display (command in a0) mv a1, t1 # data to display in a1 ecall # displays result on console procedure: li s1, 3 # using new register li t0, 1000 add a0, s1, t0 jalr zero, ra, 0 # return</pre>
Output:
3505

Task 2:

Description:

This task demonstrates how to store and retrieve register values using the stack in RISC-V. It uses the stack pointer (sp), store word (sw), and load word (lw) instructions. It helps understand stack memory allocation and register data preservation.

Code:

Code	Explanation
addi s0, zero, 5	Immediate addition of 5 + 0 and store in s0
addi t1, zero, 10	Immediate addition of 10 + 0 and store in t1
addi t0, zero, 15	Immediate addition of 15 + 0 and store in t0
addi sp, sp, -12	Initialize the stack memory, allocate 12 bytes (4 for each word)
sw s0, 0(sp)	Store/push s0 in stack
sw t0, 4(sp)	Store/push t0 in stack
sw t1, 8(sp)	Store/push t1 in stack
lw t4, 8(sp)	Load/pop the value of t1 into t4 from stack
lw t2, 4(sp)	Load/pop the value of t0 into t2 from stack
lw t3, 0(sp)	Load/pop the value of s0 into t3 from stack
addi sp, sp, 12	Release the stack space

Task 3:

Description:

This task implements an equation solver using a separate procedure in RISC-V assembly. The procedure calculates the product of the sum and difference of two numbers, $((a+b)(a-b))$. It demonstrates passing control to a procedure and returning the calculated result.

Code:

```
.text
main:
addi a0,x0,0
jal ra, EqSolver
j Exit
```

```
EqSolver:
addi s1, x0, 10
addi s0, x0, 5
add a1, s1, s0
sub a2, s1, s0
mul a0, a1, a2
jalr zero, ra, 0
```

Exit:

Output:

PC	Machine Code	Basic Code	Original Code		
0x0	0x00000513	addi x10 x0 0	addi a0,x0,0	s0 (x8)	5
0x4	0x008000EF	jal x1 8	jal ra, EqSolver # call EqSolver		
0x8	0x01C0006F	jal x0 28	j Exit	s1 (x9)	10
0xc	0x00A00493	addi x9 x0 10	addi s1, x0, 10		
0x10	0x00500413	addi x8 x0 5	addi s0, x0, 5	a0 (x10)	75
0x14	0x008485B3	add x11 x9 x8	add a1, s1, s0		
0x18	0x40848633	sub x12 x9 x8	sub a2, s1, s0	a1 (x11)	15
0x1c	0x02C58533	mul x10 x11 x12	mul a0, a1, a2		
0x20	0x00008067	jalr x0 x1 0	jalr zero, ra, 0 # return	a2 (x12)	5

Task 4:

Description:
This task calculates the area and perimeter of a square using two separate procedures, AreaFinder and PerimeterFinder. Both procedures are called from the main program to perform their respective calculations. It demonstrates the use of multiple procedures and returning results through registers.
Code:
<pre>.text main: addi a0,x0,0 # Area addi a1,x0,0 # perimeter jal ra, AreaFinder jal ra, PerimeterFinder j Exit AreaFinder: addi s1, x0, 10 # addi s0, x0, 2 mul a0, s0, s1 jalr zero, ra, 0 PerimeterFinder: addi s2, x0, 10 #length addi s3, x0, 4 mul a1, s2, s3 jalr zero, ra, 0 Exit:</pre>
Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00000513	addi x10 x0 0	addi a0,x0,0 # Area
0x4	0x00000593	addi x11 x0 0	addi a1,x0,0 # perimeter
0x8	0x00C000EF	jal x1 12	jal ra, AreaFinder
0xc	0x018000EF	jal x1 24	jal ra, PerimeterFinder
0x10	0x0240006F	jal x0 36	j Exit
0x14	0x00A00493	addi x9 x0 10	addi s1, x0, 10 #
0x18	0x00200413	addi x8 x0 2	addi s0, x0, 2
0x1c	0x02940533	mul x10 x8 x9	mul a0, s0, s1
0x20	0x00000067	jalr x0 x1 0	jalr zero, ra, 0
0x24	0x00A00913	addi x18 x0 10	addi s2, x0, 10 #length
0x28	0x00400993	addi x19 x0 4	addi s3, x0, 4
0x2c	0x033905B3	mul x11 x18 x19	mul a1, s2, s3
0x30	0x00000067	jalr x0 x1 0	jalr zero, ra, 0

Task 5:

Description:
This task implements a polynomial equation (x^3+5x^2+6x) using a separate procedure in RISC-V assembly. It performs multiplication and addition operations to calculate the polynomial's value. It demonstrates arithmetic operations and procedure execution.
Code:
<pre> .text main: addi a0,x0,0 # Area jal ra, EqSolver j Exit EqSolver: addi s0, x0, 4 # x = 4 addi a1, x0, 5 # a1 = 5 addi a2, x0, 6 # a2 = 6 mul s1, s0, s0 # s1 = x^2 mul s2, s1, s0 # s2 = x^3 mul s3, s1, a1 # 5*x^2 </pre>

```
mul s4, s0, a2 # 6*x
add a3, s2, s3 # a3 = x^3 + 5x^2
add a0, a3, s4 # a0 = x^3 + 5*x^2 + 6*x = 168
jalr zero, ra, 0
```

Exit:

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00000513	addi x10 x0 0	addi a0,x0,0 # Area
0x4	0x008000EF	jal x1 8	jal ra, EqSolver
0x8	0x02C0006F	jal x0 44	j Exit
0xc	0x00400413	addi x8 x0 4	addi s0, x0, 4 # x = 4
0x10	0x00500593	addi x11 x0 5	addi a1, x0, 5 # a1 = 5
0x14	0x00600613	addi x12 x0 6	addi a2, x0, 6 # a2 = 6
0x18	0x028404B3	mul x9 x8 x8	mul s1, s0, s0 # s1 = x^2
0x1c	0x02848933	mul x18 x9 x8	mul s2, s1, s0 # s2 = x^3
0x20	0x02B489B3	mul x19 x9 x11	mul s3, s1, a1 # 5*x^2
0x24	0x02C40A33	mul x20 x8 x12	mul s4, s0, a2 # 6*x
0x28	0x013906B3	add x13 x18 x19	add a3, s2, s3 # a3 = x^3 + 5x^2
0x2c	0x01468533	add x10 x13 x20	add a0, a3, s4 # a0 = x^3 + 5*x^2 + 6*x = 168
0x30	0x00000067	jalr x0 x1 0	jalr zero, ra, 0

a0 (x10) 168

Task 6:

Description:

This task implements an if-else-if-else structure using conditional branch instructions in RISC-V. Depending on the input values, it calls one of three procedures to calculate the required result. It demonstrates conditional branching and procedure calls.

Code:

```
.text
main:
    addi a0, x0, 0
    addi a1, x0, 2 # k = 1
    addi a2, x0, 3 # n = 2
    addi a3, x0, 1
    beq a1, a3, F1
    bgeu a2, a1, F2
```

j F3
j Exit

F1:

mul s0, a1, a1
mul s1, a2, a2
add a0, s0, s1
j Exit

F2:

mul s1, a2, a2
mul a0, s1, a2
j Exit

F3:

addi a0, a2, 5
jalr zero, ra, 0
j Exit

Exit:

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x0000513	addi x10 x0 0	addi a0, x0, 0
0x4	0x0200593	addi x11 x0 2	addi a1, x0, 2 # k = 1
0x8	0x0300613	addi x12 x0 3	addi a2, x0, 3 # n = 2
0xc	0x0100693	addi x13 x0 1	addi a3, x0, 1
0x10	0x0058863	beq x11 x13 16	beq a1, a3, F1
0x14	0x00867E63	bgeu x12 x11 28	bgeu a2, a1, F2
0x18	0x0240006F	jal x0 36	j F3
0x1c	0x02C0006F	jal x0 44	j Exit
0x20	0x02B58433	mul x8 x11 x11	mul s0, a1, a1
0x24	0x02C60483	mul x9 x12 x12	mul s1, a2, a2
0x28	0x00940533	add x10 x8 x9	add a0, s0, s1
0x2c	0x01C0006F	jal x0 28	j Exit
0x30	0x02C60483	mul x9 x12 x12	mul s1, a2, a2
0x34	0x02C48533	mul x10 x9 x12	mul a0, s1, a2
0x38	0x0100006F	jal x0 16	j Exit
0x3c	0x00560513	addi x10 x12 5	addi a0, a2, 5
0x40	0x00008067	jalr x0 x1 0	jalr zero, ra, 0
0x44	0x0040006F	jal x0 4	j Exit

Task 7:

Description:

This task implements conditional statements to select one of three functions based on the values of n and k. The selected function calculates either the square, cube, or incremented value of n. It demonstrates decision-making using branch instructions and function calls.

Code:

```
.text
main:
    addi a0, x0, 0
    addi a1, x0, 5    # n = 5
    addi a2, x0, 0    # k = 0
    beq a2, x0, F1    # if (k == 0)
    beq a1, a2, F2    # else if (n == k)
    jal ra, F3        # else
    addi s0, a0, 0    # y = fun3(n)
    j Exit

F1:
    addi a0, a1, 0    # argument x = n
    jal ra, Fun1
    addi s0, a0, 0    # y = fun1(n)
    j Exit
```

F2:

```
addi a0, a1, 0    # argument z = n
jal ra, Fun2
addi s0, a0, 0    # y = fun2(n)
j Exit
```

F3:

```
addi a0, a1, 0    # argument w = n
jal ra, Fun3
addi s0, a0, 0    # y = fun3(n)
j Exit
```

fun1(x) = x * x

Fun1:

```
mul a0, a0, a0
jalr zero, ra, 0
```

fun2(z) = z * z * z

Fun2:

```
mul t0, a0, a0
mul a0, t0, a0
jalr zero, ra, 0
```

fun3(w) = w + 1

Fun3:

```
addi a0, a0, 1
jalr zero, ra, 0
```

Exit:

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x0000513	addi x10 x0 0	addi a0, x0, 0
0x4	0x00500593	addi x11 x0 5	addi a1, x0, 5 # n = 5
0x8	0x00000613	addi x12 x0 0	addi a2, x0, 0 # k = 0
0xc	0x00000A63	beq x12 x0 20	beq a2, x0, F1 # if (k == 0)
0x10	0x02C58063	beq x11 x12 32	beq a1, a2, F2 # else if (n == k)
0x14	0x02C000EF	jal x1 44	jal ra, F3 # else
0x18	0x00050413	addi x8 x10 0	addi s0, a0, 0 # y = fun3(n)
0x1c	0x0500006F	jal x0 80	j Exit
0x20	0x00058513	addi x10 x11 0	addi a0, a1, 0 # argument x = n
0x24	0x02C000EF	jal x1 44	jal ra, Fun1
0x28	0x00050413	addi x8 x10 0	addi s0, a0, 0 # y = fun1(n)
0x2c	0x0400006F	jal x0 64	j Exit
0x30	0x00058513	addi x10 x11 0	addi a0, a1, 0 # argument z = n
0x34	0x024000EF	jal x1 36	jal ra, Fun2
0x38	0x00050413	addi x8 x10 0	addi s0, a0, 0 # y = fun2(n)
0x3c	0x0300006F	jal x0 48	j Exit
0x40	0x00058513	addi x10 x11 0	addi a0, a1, 0 # argument w = n
0x44	0x020000EF	jal x1 32	jal ra, Fun3
0x48	0x00050413	addi x8 x10 0	addi s0, a0, 0 # y = fun3(n)
0x4c	0x0200006F	jal x0 32	j Exit
0x50	0x02A50533	mul x10 x10 x10	mul a0, a0, a0
0x54	0x00008067	jalr x0 x1 0	jalr zero, ra, 0
0x58	0x02A502B3	mul x5 x10 x10	mul t0, a0, a0
0x5c	0x02A28533	mul x10 x5 x10	mul a0, t0, a0
0x60	0x00008067	jalr x0 x1 0	jalr zero, ra, 0
0x64	0x00150513	addi x10 x10 1	addi a0, a0, 1
0x68	0x00008067	jalr x0 x1 0	jalr zero, ra, 0

Task 8a:

Description:
This task implements the Fibonacci sequence in RISC-V assembly using two approaches: iterative execution and recursion. The iterative approach uses a loop, while the recursive approach involves repeated function calls and stack operations. It demonstrates loops, recursive procedures, stack management, and return values.
Code:
<pre> .text main: addi a0, x0, 0 addi a1, x0, 2 # n = ? jal ra, Fib j Exit </pre>

Fib:

```
addi s0, x0, 0 # a = 0
addi s1, x0, 1 # b = 1
addi s2, x0, 2 # c = 2
beq a1, x0, Re1
addi s3, x0, 2 # i = 2
addi a2, a1, 1
```

Loop:

```
bge s3, a2, Re2
add s2, s0, s1 # c = a + b
addi s0, s1, 0 # b = a
addi s1, s2, 0 # b = c
addi s3, s3, 1 # i++
j Loop
```

Re1:

```
add a0, x0, s0
jalr zero, ra, 0
```

Re2:

```
add a0, x0, s2
jalr zero, ra, 0
```

Exit:

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00000513	addi x10 x0 0	addi a0, x0, 0
0x4	0x00800593	addi x11 x0 8	addi a1, x0, 8 # n = ?
0x8	0x008000EF	jal x1 8	jal ra, Fib
0xc	0x0440006F	jal x0 68	j Exit
0x10	0x00000413	addi x8 x0 0	addi s0, x0, 0 # a = 0
0x14	0x00100493	addi x9 x0 1	addi s1, x0, 1 # b = 1
0x18	0x00200913	addi x18 x0 2	addi s2, x0, 2 # c = 2
0x1c	0x02050263	beq x11 x0 36	beq a1, x0, Re1
0x20	0x00200993	addi x19 x0 2	addi s3, x0, 2 # i = 2
0x24	0x00150613	addi x12 x11 1	addi a2, a1, 1
0x28	0x02C00063	bge x19 x12 32	bge s3, a2, Re2
0x2c	0x00940933	add x18 x8 x9	add s2, s0, s1 # c = a + b
0x30	0x00040413	addi x8 x9 0	addi s0, s1, 0 # b = a

0x34	0x00090493	addi x9 x18 0	addi s1, s2, 0 # b = c
0x38	0x00198993	addi x19 x19 1	addi s3, s3, 1 # i++
0x3c	0xFEDFF06F	jal x0 -20	j Loop
0x40	0x00800533	add x10 x0 x8	add a0, x0, s0
0x44	0x00000867	jalr x0 x1 0	jalr zero, ra, 0
0x48	0x01200533	add x10 x0 x18	add a0, x0, s2
0x4c	0x00000867	jalr x0 x1 0	jalr zero, ra, 0

Task 8b:

Description:

This task implements the Fibonacci sequence in RISC-V assembly using two approaches: iterative execution and recursion. The iterative approach uses a loop, while the recursive approach involves repeated function calls and stack operations. It demonstrates loops, recursive procedures, stack management, and return values.

Code:

```
.text
main:
    addi a0, x0, 0
    addi a1, x0, 5    # n = 5
    jal ra, Fib
    addi s0, a0, 0    # y = fib(n)
    j Exit

Fib:
    beq a1, x0, Re0 # if (n == 0)
    addi t0, x0, 1 # else if (n == 1)
    beq a1, t0, Re1
    addi sp, sp, -16 # Save registers and return address
    sw ra, 12(sp)
    sw a1, 8(sp)
    addi a1, a1, -1 # fib(n-1)
    jal ra, Fib
    sw a0, 4(sp) # Save fib(n-1)
    lw a1, 8(sp) # fib(n-2)
    addi a1, a1, -2
    jal ra, Fib
    lw t0, 4(sp) # fib(n-1) + fib(n-2)
    add a0, t0, a0
    lw ra, 12(sp) # Restore return address and stack
    addi sp, sp, 16

    jalr zero, ra, 0
```

Re0:

```
addi a0, x0, 0  
jalr zero, ra, 0
```

Re1:

```
addi a0, x0, 1  
jalr zero, ra, 0
```

Exit:

Output: